***Deepak Advani***
***d20A/ 01***
***Experiment-1***

**Aim:** Cryptography in Blockchain, Merkle root Tree Hash

**Tasks Performed:**

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

**Program :**

```
import hashlib

data = input("Enter data: ")


hash_value = hashlib.sha256(data.encode()).hexdigest()

print("\nInput Data:", data)
print("SHA-256 Hash:", hash_value)
```

**Output:**

```
Enter data: DEEPAK

Input Data: DEEPAK
SHA-256 Hash:
d26385556e044ab722782534ca30811173fab339fe309d3f0e39421ff70cc01a
```

2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

**Program :**

```
import hashlib
data = input("Enter data: ")
nonce = int(input("Enter nonce: "))
input_data = data + str(nonce)
hash_value = hashlib.sha256(input_data.encode()).hexdigest()
```

```
print("\nInput Data:", data)
 print("Nonce:", nonce)
print("Combined Data:", input_data)
print("SHA-256 Hash:", hash_value)
```

**Output:**

```
Enter data: DEEPAK
Enter nonce: 10

Input Data: DEEPAK
Nonce: 10
Combined Data: DEEPAK10
SHA-256                                                    Hash:
   6b06f6a3b77abced35a99b27dd7290716444cc00d9ebe2c8f1d44d0f1a8f98
   ad
```

3. **Proof-of-Work Puzzle Solving:**

   o Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

**Program :**

```
import hashlib

data = input("Enter data: ")

difficulty = int(input("Enter difficulty (number of leading
   zeros): "))


target = "0" * difficulty

nonce = 0

while True:


    input_data = data + str(nonce)

    hash_value = hashlib.sha256(input_data.encode()).hexdigest()
     if hash_value.startswith(target):
         print("\nGolden Nonce Found!")
```

```
            print("Data:", data)
            print("Golden Nonce:", nonce)
            print("Hash:", hash_value)
            break

        nonce += 1
```

**Output:**

```
Enter data: DEEPAK
Enter difficulty (number of leading zeros): 4

Golden Nonce Found!
Data: DEEPAK
Golden Nonce: 53693
Hash:
   0000f925166215f4043796524905de184d45e43345b5807ff2d9ec815bbc9a
   76
```

4. **Merkle Tree Construction:**

   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

**Program :**

```
import hashlib
   def sha256(data):
          return hashlib.sha256(data.encode()).hexdigest()
   transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Roo -> Bob : $50"
]
print("=====================================")
print("STEP 1: TRANSACTION HASHES")
print("=====================================")
current_level = []
    for transaction in transactions:

    transaction_hash = sha256(transaction)
```

```
        current_level.append(transaction_hash)

        print("\nTransaction:", transaction)
        print("Hash:", transaction_hash)


level = 1
while len(current_level) > 1:
    print("\n====================================")
    print("MERKLE TREE LEVEL", level)
    print("====================================")

    if len(current_level) % 2 != 0:
        current_level.append(current_level[-1])
        print("\nOdd number of hashes detected.")
        print("Last hash is duplicated.")
    next_level = []
    for i in range(0, len(current_level), 2):
        left_hash = current_level[i]
        right_hash = current_level[i + 1]
        combined_hashes = left_hash + right_hash
        parent_hash = sha256(combined_hashes)
        print("\nLeft Hash:")
        print(left_hash)
        print("\nRight Hash:")
        print(right_hash)
        print("\nCombined Hashes:")
        print(combined_hashes)
        print("\nParent Hash:")
        print(parent_hash)
        next_level.append(parent_hash)
    current_level = next_level

    level += 1
    merkle_root = current_level[0]
print("\n====================================")
print("FINAL MERKLE ROOT")
print("====================================")
print("Merkle Root Hash:")
```

```
        print(merkle_root)
        print("====================================")
```

**Output:**

```
====================================
STEP 1: TRANSACTION HASHES
====================================
Transaction: Alice -> Bob : $200
Hash: f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
Transaction: Bob -> Dave : $500
Hash: 768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
Transaction: Dave -> Eve : $100
Hash: 43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
Transaction: Eve -> Alice : $300
Hash: cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
Transaction: Roo -> Bob : $50
Hash: cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
====================================
MERKLE TREE LEVEL 1
====================================
Odd number of hashes detected.
Last hash is duplicated.
Left Hash:
f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760
Right Hash:
768890d3b58d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
Combined Hashes:
f817ce0bddbfa417eba4ae519a8b864ba7b1eed286e10c4b92086cc713279760768890d3b5
8d4d6a17673e6f750a0145f546557c9529d258750e1ca0cdbb5b46
Parent Hash:
471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
Left Hash:
43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890da
Right Hash:
cba341d7965fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
Combined Hashes:
43dd5729f00e01eea73a858d02f9c714c882bab801de203c1314d6318d6890dacba341d796
5fff73181a2a6579799dc9644e84380fa9e14f12ebf78c70316a8a
```

Parent Hash:
a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Left Hash:
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Right Hash:
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Combined Hashes:
cfe1dc26c7c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58accfe1dc26c7
c3e26c19aed0552bd4b9db2cc797a65a3a26316f47ea95cb1b58ac
Parent Hash:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e

==========
MERKLE TREE LEVEL 2
==========

Odd number of hashes detected.
Last hash is duplicated.
Left Hash:
471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9
Right Hash:
a61c504d72e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Combined Hashes:
471608d3d6f2a642ff3a84dfaeeaebb1c271a9aed5b37a82d61c4784558e80e9a61c504d72
e6ce69b2ec8791ff7469cc004704f727f8dbbf420fadbc4e4c0de7
Parent Hash:
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
Left Hash:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Right Hash:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Combined Hashes:
314ca746b3dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e314ca746b3
dafe63a1e0b939ec3042df6270de7703aa0eb7e699a3f2c2e7c18e
Parent Hash:
6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45

```
======================================
MERKLE TREE LEVEL 3
======================================
Left Hash:
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee
Right Hash:
6bb2c8b4fd23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
Combined Hashes:
ae847393c3dff60e1d95cfdbb1c1c886d0bf7cbd26410bcc2469e023e94cb8ee6bb2c8b4fd
23658c0b716069761d9811ced49f7e4f2f736b493fe0c0a96b4f45
Parent Hash:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

======================================
FINAL MERKLE ROOT
======================================
Merkle Root Hash:
96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0
======================================
```

Conclusion:

This study demonstrates the importance of SHA-256 hashing, nonce-based Proof-of-Work, and Merkle Trees in blockchain technology. These cryptographic techniques ensure data integrity, secure mining, and efficient transaction verification, forming the foundation of blockchain security and reliability.